

DPI343

Computer Organization and Assembly Language

Lecture 8

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

What we already know from the previous lecture

Instructions CALL and RET

Simple explanation

Both instructions are similar to unconditional JMP, except that CALL stores the return address on the stack and RET takes the return address from the top of the stack.

CALL *label*

- 1) Address of the next instruction that follows the CALL instruction (i.e. return address) is pushed onto top of the stack
- 2) Control is transferred to the *label*

RET [n]

- 1) Address is retrieved from the top of the stack (like pop)
- 2) Control is transferred to this address
- 3) If optional operand **n** is provided then top of the stack is adjusted: **ESP = ESP + n**

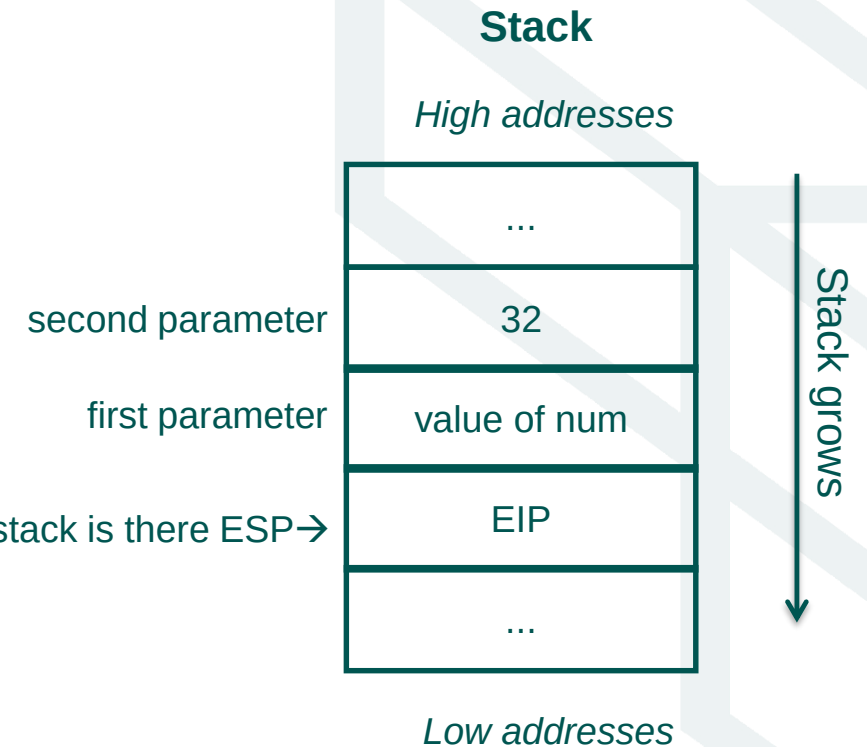
Passing parameters via stack

```
.data
num  dword  -2
buff byte  11 dup (' '), 0
```

```
.code
:::
    push 32      ; 2nd parameter
    push num     ; 1st parameter
    call Ones
:::
```

```
Ones proc near
    . . .
    ret 8
Ones endp
```

At the entry to procedure *Ones* top of the stack is there ESP→



Standard calling convention (stdcall) requires that arguments are pushed into stack from right to left (i.e. the last parameter is pushed first) and the called procedure must clean up the stack

Passing parameters via stack (cont.)

Ones proc near stdcall

;Parameters: 1)doubleword value, 2)doubleword - number of rightmost bits to count

;Return value: result in EAX

```
push ebp          ; save EBP
```

```
mov  ebp, esp     ; save ESP to EBP
```

```
push ebx          ; save EBX
```

```
push ecx          ; save ECX
```

```
mov  ecx, [ebp+12] ;ECX=second parameter
```

```
mov  eax, [ebp+8]  ;EAX=first parameter
```

```
xor  ebx, ebx
```

testb:

```
test eax, 1
```

```
jz   skip
```

```
inc  ebx
```

skip:

```
shr  eax, 1
```

```
loop testb
```

```
mov  eax, ebx      ; copy result to EAX
```

```
pop  ecx           ; restore ECX
```

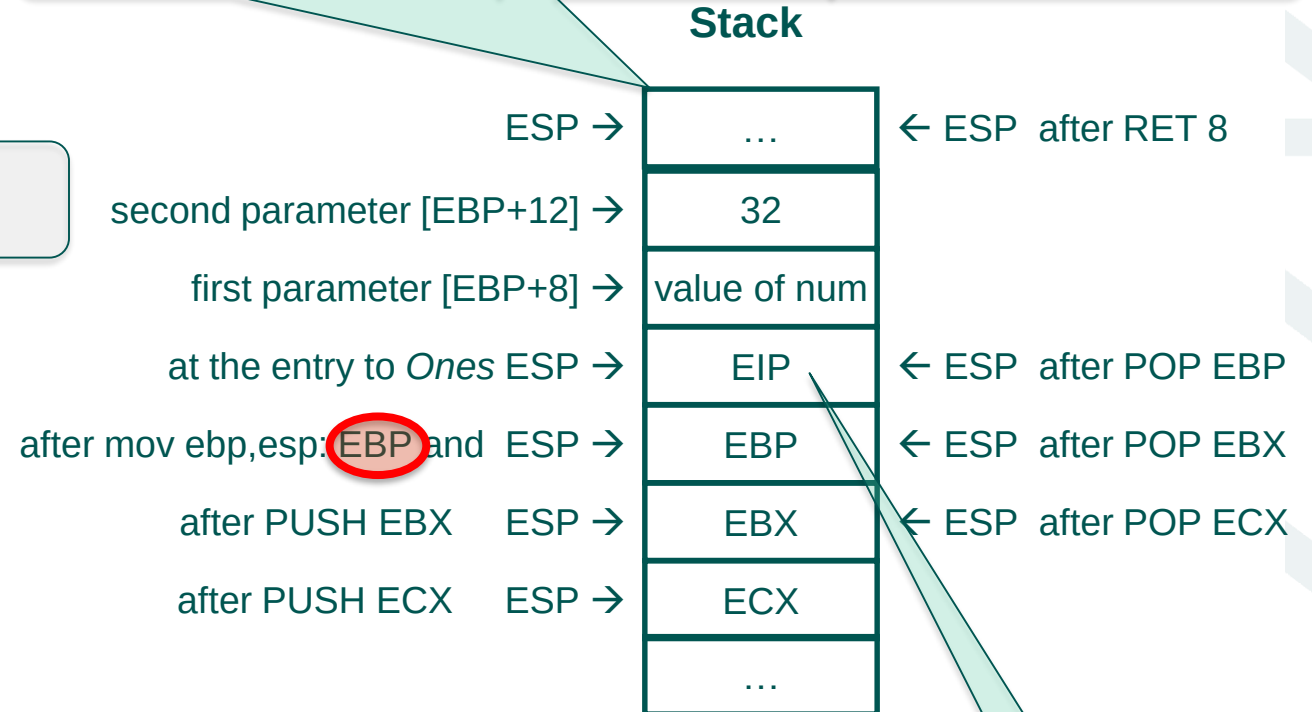
```
pop  ebx           ; restore EBX
```

```
pop  ebp           ; restore EBP
```

```
ret  8 ;return and adjust top of the stack
```

Ones endp

After returning to the calling procedure, the top of the stack is where it was before the parameters were pushed onto the stack



NEAR procedure call pushes only EIP onto stack

In STDCALL procedure must adjust top of the stack

Procedures (continued)

- Building executable code from separately compiled modules
- Calling conventions revisited
- Decorated names
- Some side effects and how to deal with them
- Combining code written in different languages
 - call Assembly procedure from C++
 - call C++ function from Assembly

Using external procedures. Version 1

File ones1.asm:

```
.586
.model flat
.code
Ones proc
; . . .
Ones endp
end
```

File mainones1.asm:

```
.586
.model flat

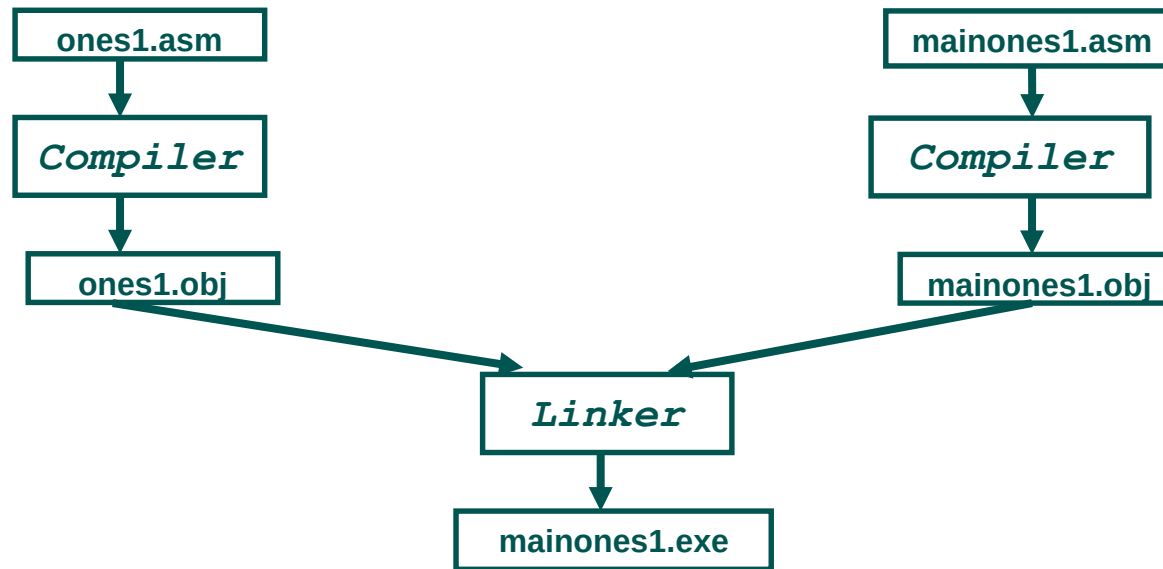
extern  Ones:PROC

.code
main    proc
        push 32      ; second parameter
        push num     ; first parameter
        call Ones
; . . .
main    endp
end     main
```

Directive EXTERN to declare external name Ones

Call procedure Ones

How to build executable



To build executable:

1) compile each .asm file

```
m1 /c ones1.asm
```

```
m1 /c mainones1.asm
```

2) build mainones1.exe using linker

```
link /subsystem:console mainones1 ones1
```

Default name of the executable file is name of the first .obj file in the list.
You may use linker switch **/OUT:name** to set another file name

Calling conventions (revisited)

Be careful with external names the compiler automatically generates depending on the specified calling convention. They may differ from the names you typed in the source code.

How to see external names created by the compiler?

- 1) Linker error messages will contain unresolved external names...
- 2) You may use the utility **dumpbin** with the **/symbols** switch to see external names created by the compiler:

```
dumpbin /symbols mainones1.obj
```

This is why we didn't use any calling convention references in PROC directives of our first example:

```
Ones proc
and
main proc
```

So all external names in the .obj files were kept exactly as the programmer had written them.

```
C:\COAL\COAL2025_32\MyAsm>dumpbin /symbols mainones1.obj
Microsoft (R) COFF/PE Dumper Version 14.44.35217.0
Copyright (C) Microsoft Corporation. All rights reserved.

Dump of file mainones1.obj

File Type: COFF OBJECT

COFF SYMBOL TABLE
000 01038991 ABS      notype      Static      | @comp.id
001 00000010 ABS      notype      Static      | @feat.00
002 00000000 SECT1   notype      Static      | .text$mn
      Section length 31, #relocs 7, #linenums 0, checksum 0
004 00000000 SECT2   notype      Static      | .data
      Section length 10, #relocs 0, #linenums 0, checksum 0
006 00000000 SECT3   notype      Static      | .debug$S
      Section length 78, #relocs 0, #linenums 0, checksum 0
008 00000000 SECT4   notype      Static      | .directive
      Section length 3A, #relocs 0, #linenums 0, checksum 0
00A 00000000 UNDEF   notype      External     | _puts
00B 00000000 UNDEF   notype      External     | __ltoa
00C 00000000 UNDEF   notype      External     | _ExitProcess@4
00D 00000000 UNDEF   notype      External     | Ones
00E 00000000 SECT2   notype      Static      | num
00F 00000004 SECT2   notype      Static      | buff
010 00000000 SECT1   notype      External     | main
```

Calling conventions (revisited)

Be careful with external names ... (continued)

When we compiled mainones1.asm, we had a warning about the entry point name:

```
C:\COAL\_COAL2025_32\MyAsm>ml /c mainones1.asm
Microsoft (R) Macro Assembler Version 14.44.35217.0
Copyright (C) Microsoft Corporation. All rights reserved.

Assembling: mainones1.asm
mainones1.asm(41) : warning A4023:with /coff switch, leading underscore required
for start address : main
```

To avoid the warning you can specify the name of the entry point not in the object file, but in the /ENTRY:name switch to the linker:

File mainones1.asm

```
main    proc
. . .
main    endp
end
```

Entry point not specified in the END directive of the source file

```
link /subsystem:console /entry:main mainones1 ones1
```

But of course you can follow the linker's suggestion to replace the entry point name and add an underscore in front of it.

Decorated names

Microsoft Learn:

"A *decorated name* is an encoded string created by the compiler during compilation of an object, data, or function definition. It records calling conventions, types, function parameters and other information together with the name. This name decoration, also known as *name mangling*, helps the linker find the correct functions and objects when linking an executable."

"Normally, you don't have to know the decorated name to write code that compiles and links successfully. Decorated names are an implementation detail internal to the compiler and linker."

For example, decorated names help distinguish C++ class methods by their parameter and return value types.

"Name decoration is also important when linking to code written in other programming languages or using other compilers. Different compilers use different name decoration conventions."

If **stdcall** convention is specified in the Assembly **EXTERN** directive, then Microsoft compiler **ml** will add leading underscore to the external name.

[File mainones1.asm:](#)

```
.586
.model flat
extern stdcall Ones:PROC
.code
main1 proc
    push 32      ; second parameter
    push num     ; first parameter
    call Ones
```

00D	00000000	UNDEF	notype	External	_Ones
00E	00000000	SECT2	notype	Static	num
00F	00000004	SECT2	notype	Static	buff
010	00000000	SECT1	notype ()	External	main

To successfully build an executable with the linker, there must be a matching external name in the procedure Ones .obj file.

Decorated names (continued)

You may try to specify stdcall convention in the Ones source code:

[File ones1.asm](#)

```
Ones proc stdcall
```

```
. . .
```

```
Ones endp
```

```
end
```

And use **dumpbin** to see external name created by the compiler:

```
dumpbin /symbols ones1.obj
```

```
008 00000000 SECT1 notype () External [ _Ones@0
```

If you try to link, there will be an error because the external names unfortunately do not match!
Name **_Ones** is required in mainones1.obj, but **_Ones@0** is provided in ones1.obj

Why it is so?

If **stdcall** is specified in the **PROC** directive of the procedure then compiler will add leading underscore and also an @ symbol with the total number of bytes in the parameters to be passed on the stack.

But actually we are passing 2 parameters on the stack (total 8 bytes)!

We could specify this information in the PROC directive in ones1.asm: **Ones proc stdcall num:dword, n:dword**

But then the external name **_Ones@8** will be created in the ones1.obj ...

It seems like this is all getting messy and hard to organize...

Some side effects and how to deal with them

In the next version, we will use our knowledge of more convenient procedure calling using the PROTO and INVOKE directives. First, the calling procedure must describe the procedure to be called in the PROTO directive. Then, the procedure can be called either with the usual CALL instruction (having previously pushed the parameters onto the stack), or using the INVOKE directive.

[File mainones2.asm](#)

```
.586
.model flat

. . .
Ones      PROTO stdcall num:dword,n:dword ;Prototype for the procedure Ones
. . .
.code
main      proc
            push 32      ; second parameter
            push num     ; first parameter
            call Ones    ; result is in EAX
. . .
            invoke  Ones,num,32
. . .
main      endp
end main
```

If you'll try **dumpbin** for mainones2.obj:

00D 00000000	UNDEF	notype	()	External	<u>Ones@8</u>
00E 00000000	SECT2	notype		Static	num

Some side effects and how to deal with them (continued)

Now write a new version of the Ones procedure:

[File ones2.asm](#)

```
.586
.code
Ones proc stdcall num:dword, n:dword
    push ebp    ; save registers
    mov  ebp, esp
    . . .
Ones endp
end
```

If you'll try the **dumpbin** utility for ones2.obj:

```
008 00000000 SECT1  notype ()  External  | _Ones@8
```

Now we have matching external names in mainones2.obj and ones2.obj

Lets build mainones2.exe: `link /subsystem:console mainones2 ones2`

No errors 😊

Lets run mainones2.exe

```
C:\COAL\_COAL2025_32\MyAsm>mainones2
6
8
```

This is a wrong result! ☹

Some side effects and how to deal with them (continued)

Lets run mainones2.exe in the WinDbg debugger.

At the beginning of procedure Ones we will see incorrect executable code!

Instructions

```
push ebp
```

```
mov    ebp, esp
```

are duplicated 2 times even though they are correct in the source code:

```
Ones proc stdcall num:dword, n:dword
```

```
push ebp           ; save registers
```

```
mov    ebp, esp
```

push ebx

push ecx

```
mov ecx, [ebp+12] ; ecx = second parameter
```

```
mov    eax, [ebp+8]    ; eax = first param
```

```
mainones2!0nes:
```

00eb10a0 55

```
00eb10a1 8bec
```

```
00eb10a3 55
```

```
00eb10a4 8bec
```

```
00eb10a6 53
```

```
00eb10a7 51
```

```
00eb10a8 8b4d0c
```

```
00eb10ab 8b4508
```

push ebp

```
mov     ebp, esp
```

push ebp

```
mov     ebp, esp
```

```
push    ebx
```

push ecx

```
mov     ecx,dword ptr [ebp+0Ch]
```

```
mov     eax,dword ptr [ebp+8]
```

This is because the **ml** compiler automatically generates the procedure prolog code based on the information in the PROC directive.

Of course, you can use this automation, but it is better to write all the necessary code yourself to fully understand how it works. At least when you start learning assembly.

See more about the options of the PROC directive, prologue and epilogue code in the documentation.

Some side effects and how to deal with them (continued)

To avoid the automatically generated prologue and epilogue code, add two OPTIONS directives.

[File ones2.asm](#)

```
.586
```

```
.code
```

```
option prologue:none  
option epilogue:none
```

```
Ones proc stdcall num:dword, n:dword
```

```
    push ebp
```

```
    mov  ebp, esp
```

```
    . . .
```

```
Ones    endp
```

```
end
```

What is not covered

- Sometimes stack memory is used to store local variables of procedures.
- Since the stack also contains received parameters, the procedure prologue and epilogue code must be made a bit more complicated and care must be taken to release the stack memory allocated to local variables.
- Not forgetting to align the top of the stack according to the calling convention.

We will not cover this in this course, but those who are interested will find information in the documentation or AoA.

Assembly procedures and C/C++

- The Visual C++ compilers allow you to specify conventions for passing arguments and return values between functions and callers.
- Not all conventions are available on all supported platforms, and some conventions use platform-specific implementations.
- On x86 platforms, all arguments are widened to 32 bits when they are passed.
- Parameters of C/C++ function are pushed onto the stack from right to left.
- Return values are also widened to 32 bits and returned in the EAX register, except for 8-byte structures, which are returned in the EDX:EAX register pair.
- Larger structures are returned in the EAX register as pointers to return structures.

Assembly procedures and C/C++

Example 1: calling C++ functions from Assembly code

Try MS Visual Studio project **asmones_mainasm** and run the code using debugger

Example 2: calling Assembly code from C++ functions

Try MS Visual Studio project **asmones_maincpp** and run the code using debugger

Thank You!

